

# Speech - Text only

## SLIDE 1 - Title

0:00 → 0:30

Good morning everyone, thank you for being here. My name is Filippo Del Ministro and today I'll present my thesis, "An Incremental Approach to Automatic Firewall Configuration," supervised by professors Fulvio Valenza, Daniele Brighenti, Francesco Pizzato, and Riccardo Sisto.

## SLIDE 2 - Background

0:30 → 1:45

Let's start with the context. Modern networks are complex, and the growing complexity is the reason why security is shifting from a single perimeter firewall to a distributed approach, namely micro segmentation.

This shift brings well-known benefits: finer-grained control over traffic, containment of lateral movement if one segment is compromised, and policies tailored to each part of the network — for instance, a stricter policy for the data centre than for a general office segment. But it also complicates configuration: there are more firewall instances to manage, and if that configuration is kept manual, it becomes error-prone and offers no guarantee of correctness — a single mistake can silently violate a security requirement, or block legitimate traffic.

## SLIDE 3 - VEREFOO

1:45 → 3:15

In this context, help network administrators manage their network without configuration errors. VEREFOO is an automated tools for firewall placement and configuration

VEREFOO takes two inputs: the network topology, formally described through a Service Graph — its nodes and links — and a set of Network Security Requirements, the NSRs, which describe which traffic flows must be allowed or denied

From these, it produces two outputs: a firewall allocation scheme, meaning where the firewalls are placed in the topology, and a filtering policy for each firewall, meaning how to configure it to properly enforce the security requirements.

VEREFOO guarantees optimality — minimizing the number of firewalls and configuration rules — and formal correctness: every requirement is guaranteed to be enforced.

The main limitation of a tool built this way is that it doesn't handle change well: modern networks are dynamic, with nodes added and removed and traffic requirements evolving continuously. The original version of VEREFOO does not address this problem at all, re-computing the new configuration from scratch every time a novel security requirement is added,

## SLIDE 4 - React-VEREFOO

3:15 → 4:15

VEREFOO, as we just saw, guarantees correctness and optimality by construction, but it's monolithic: any change to the NSR set triggers a full recomputation from scratch.

To accommodate the dynamism of real networks, React-VEREFOO was developed: it takes an existing configuration already in place, the initial security requirements, plus a set of additional requirements, and reconfigures only the area of the network that actually needs to change.

To do this, React-VEREFOO classifies each NSR as to-be-added, to-be-removed, or kept, and only reconfigures the firewalls that fall within the affected area. Formal correctness is still guaranteed

What changes is performance. React is much faster compared to the monolithic version, which always compute the configuration from scratch.

Experimental tests, however, show that this benefit is not constant and depends on the number of additional requirements to be added. In particular, React it's much faster when only a small portion of the requirements changes, and it is comparable to the monolithic run, whenever the batch of new requirements is larger.

It's exactly on this last observation that the key intuition of this research is built — which brings us to the next slide.

## **SLIDE 5 - Research Question**

3:15 → 4:15

The key idea of this research is then to exploit this very last observation, that React-VEREFOO's performance are much faster when a small batches of new requirements is provided, to replace the one-shot approach of VEREFOO, with multiple simple calls to React-VEREFOO, with one single requirement each time.

## **SLIDE 6 - Thesis Contribution: Iterative VEREFOO**

4:15 → 6:00

This is exactly the contribution of this thesis: the Iterative VEREFOO approach. The idea is simple to state but effective in practice: instead of passing React-VEREFOO a batch of new requirements, we introduce them one at a time, invoking React-VEREFOO internally at each step to compute the minimal update to the current configuration. To do this, the algorithm maintains two structures that evolve across iterations: `consideredProps`, the set of already-enforced requirements, passed as the initial set to the next step; and `currentGraph`, the network configuration updated after the most recently verified requirement. This way, every iteration always receives the smallest possible incremental problem — just one new requirement on top of an already stable, verified configuration. And, crucially, correctness is always guaranteed: every NSR in the final set is satisfied by the resulting configuration.

## **SLIDE 6 - Structural Comparison**

6:00 → 6:45

The idea behind this thesis is to push the incremental approach to its extreme: instead of feeding React-VEREFOO a batch of new requirements, we give it exactly one at a time. And to be clear, we're not reinventing the wheel here — we're reusing React-VEREFOO exactly as it is, just using specific input to it.

As a quick reminder, React-VEREFOO normally takes two batches: a set of requirements already enforced, and a set of new requirements to add. What the iterative approach does is manipulate exactly this interaction. We call `P0` the initial set of requirements, and `A1`, `A2`, `A3`, and so on the additional requirements we want to introduce, one by one. So at the first iteration, `P0` is passed as the 'old' requirements and `A1` as the 'new' one. At the second iteration, `P0` plus `A1` becomes the 'old' set, and `A2` is the only 'new' requirement. And this continues for all the additional requirements (`N`).

As you can see, only one requirement at a time is introduced. Every iteration thus always receives the smallest possible incremental problem.

Correctness is always guaranteed by React-VEREFOO itself.

## **SLIDE 8 - Implementation**

6:45 → 7:45

The iterative variant was implemented entirely by modifying the `Serializer`, one of the main component of the tool — without touching anything on the caller side. Only one wrapper was added: its job is to intercept the call before it

reaches the solver, and manipulate what gets passed to React-VEREFOO, feeding it one requirement at a time instead of the full batch. Across iterations, the intermediate results are propagated through two structures that get updated at every step: `consideredProps`, tracking what's already enforced, and `currentGraph`, the configuration as it evolves. Everything else — the MaxSMT encoding, `VerefooProxy`, the Translator — is left completely untouched, which guarantees retro-compatibility with the rest of the pipeline.

## **SLIDE 9 - Validation: CESNET Topology**

7:45 → 9:00

The iterative version was validated qualitatively on the CESNET topology, a real academic network in the Czech Republic, with 23 nodes and 28 links. The initial configuration had ten requirements, already enforced by three firewalls. Starting from there, I introduced five additional requirements one at a time, tracing each iteration step by step to confirm that every new requirement was satisfied — and, just as important, that none of the previously satisfied ones were ever broken along the way.

The final configuration ends up with five firewalls in total, enforcing all fifteen requirements: the ten initial ones plus the five that were added incrementally.

## **SLIDE 10 - Validation: Iterative vs Monolithic**

9:00 → 10:00

To put this result in context, I ran the same test with vanilla VEREFOO — the monolithic version — over the exact same topology and the same set of fifteen NSRs. The result is a very similar configuration: five firewalls, three of which serve the exact same purpose, in the exact same positions, as in the iterative solution.

The difference shows up on the other two firewalls, and it is due to visibility of requirement at each step. The iterative approach never sees beyond the single requirement it's currently processing, and needs to commit to a firewall allocation at every stage. The monolithic approach, on the other hand, sees all fifteen requirements at once and can optimize globally. In this specific case, the firewall allocation is the same, but the iterative version needs to configure one extra rule.

That said, the iterative approach has a compensating advantage: it never relocates a firewall that's already valid, avoiding both the operational cost and the transient insecure states that a full re-run would cause. This matches how requirements actually accumulate over time in real networks, and it's exactly what motivates using this approach in highly dynamic environments.

## **SLIDE 11 - Conclusions & Future Work**

10:00 → 11:15

The iterative approach was validated qualitatively and reaches a result that almost matches the monolithic baseline in firewall count, with just one extra rule, despite never having visibility beyond the single requirement being processed. I expect scalability to be the main advantage as topologies and reconfigurations grow larger. Future work: quantitative time benchmark at larger scale, traffic-based requirement ordering to reduce tie-driven reconfigurations, support for NSR removal.